

MPI programming.

- For communication b/w computers in a cluster.

(Message Passing Interface). MPI is postal service standard, not a product itself.

- ⇒ Each machine in a cluster has ^{processes which have their own} mem and threads
- ⇒ Synchronization and data movement.

- MPI is SPMD

→ A communicator is a group of processes that talk to each other. MPI-COMM-WORLD, each process get a rank.

MPI-Init,

- Finalize
- Comm-size
- rank
- Send
- Recv

• mpicc main.c -o main

• mpiexec -n 16 ./main (runs 16 processes locally)

• Traditionally we have one master and multiple slaves. (at least 2)

→ just a convention

Master → slave tag 0

Slave → Master tag 1

master : 2 // can run upto 2 processes

slave1 : 3 // can run upto 3 processes.

“

slaveN : 3

- Local.

MPI-Init(&argc, &argv);

// do smthg, all processes do it

MPI-Finalize();

- For clusters we use a machinefile

so for 1 master a 2 slaves → 8 slots.

m : rank 0, 1 • master node 1

s1 : rank 2, 3, 4 • slave node 1

s2 : rank 5, 6, 7 “ 2

rank 4-7 are not run.

command is mpiexec -n (4) -f machinefile

./mc

→ total processes allowed

- Processes spawn at `MPI-INIT()`.

- `MPI-Comm-rank (MPI-COMM-WORLD, &rank)`
// get rank. ↳ int

- `MPI-Comm-size (MPI-COMM-WORLD, &size)` ↗

- In send & recv we use MPI's types coz int etc maybe diff sizes on the multiple devices.

`MPI-Get-count (&status, MPI-INT, &count)`

// How many units of type
type received so we can safely iterate (e.g 3 used in 100 buffer)

`MPI-Probe (0, 0, MPI-COMM-WORLD, &status)`
// peek msg w/o receiving it

`MPI-Get-count (&status, MPI-INT, &count)`
// see how big it is type

Comprehensive code example.

- argc and argv are necessary btw.

```
int main (int argc, char* argv[]) {  
    int rank, size;    bool active[N] = {True, ..., True};  
                        N is (size-1)  
    MPI-Init (&argc, &argv);  
    MPI-Comm-rank (MPI-COMM-WORLD, &rank);  
    MPI-Comm-size (MPI-COMM-WORLD, &size);  
    if (rank == 0) {  
        int active_slaves = size-1;  
        while (active_slaves > 0) {  
            for (int slaves=1; slaves < size; slaves++) {  
                int num = // some random num;  
                if (!active[slaves-1]) continue;
```

TAG-OK is 0
TAG-STOP is 1

```
MPI_Send(&num, 1, MPI_INT, slave, TAG,
        MPI_COMM_WORLD); }
for (int slaves=1; slaves < size; slaves++) {
    int result;
    MPI_Status stat;
    MPI_Recv(&result, 1, MPI_INT, slave, MPI_ANY_TAG,
            MPI_COMM_WORLD, &stat);
    // stopping conditions, checks if slave has
    // stopped, if yes then stops/removes it.
    if (stat.MPI_TAG == TAG_STOP) {
        active_slaves --; → active[slaves-1] = 0;
    } // slave stopped itself and sent stop tag.
    if (status result % 18 == 0) {
        MPI_Send(&result, 1, MPI_INT, slave,
        active_slaves-1 == 0; TAG_STOP, MPI_COMM_WORLD);
        active_slaves --; }
        // here we send the stop tag so
        // it can stop itself.
    }
}

else { // rank != 0
    while (1) {
        int num; MPI_Status stat;
        MPI_Recv(&num, 1, MPI_INT, 0, MPI_ANY_TAG,
            MPI_COMM_WORLD, &stat);
        if (stat.MPI_TAG == TAG_STOP)
            break;
        if (num % 2 != 0) {
```



```

        MPI_Send(&num, 1, MPI_INT, 0, TAG_STOP,
                MPI_COMM_WORLD);
        break; }
    int result = num * 2;
    MPI_Send(&result, 1, MPI_INT, 0, TAG_CHK_MPI);
}
MPI_Finalize();
return 0; }

```

- What is basically being done here is :
 - Master sends random nums to ~~integer~~ ^{active} slaves till we run out of slaves (based on conditions). The slaves send the number * 2 back.
 - if the num sent by master is odd, the slave tells master "I'm stopping" and brakes/stops.
 - if the num is multiple of 18, the master tells that slave, you should stop and the slave reads the msg & stops.
- The active array is there so we don't send data to a stopped slave.

Non-blocking version: Most is same except

- we will have ² request arrays (one for Isend and one for Irecv).
- Waitall will be called after every Isend and Irecv loop in master and wait after send & recv in slaves.

∴ MPI_Waitall(num-slaves, req, statuses)

↑ ↑
 arrays

→ So far we did point to point, now collective.

① Communication: (Group announcement etc)

- Every process calls the same collective operation.
- All collective operations are blocking.
- Broadcast: ^{root} process sends data to all

MPI_Bcast (data, count, type, root, comm)
↳ every process must call it, happens at same time (roughly). root sends, all others receive.

• Scatter:

MPI_Scatter (send-buff, send-count, send-type, recv-buff, recv-count, recv-type, root, comm)

- send-buff : array to split
 - send-count : num. of elements to send to each process.
 - recv-buff : buffer to recv chunk
 - recv-count : no. of elements to recv
- must be eq.

④ The send-buff is evenly divided to all processes. Thus the sendbuff must have
(no. of processes) × (send-count)

- only root's send-buff must be the proper array, all others can have null in it.

⇒ Same calling method, ^{as bcast} must be called by all of the processes.

- root also gets its chunk e.g.

root → 0 1,2

1 3,4

2 5,6

3 7,8

└──────────┘
 ↓
 recv buffs
case 1

0 1,2

1 3,4

root → 2 5,6

3 7,8

└──────────┘
 ↓
 recv buffs.
case 2

- chunks are assigned in rank order.

⇒ Scatterv is similar, it just allows flexibility.

MPI_Scatterv (sendbuffs, sendcounts, displs, sendbuffs)

- sendcounts ⇒ array telling count of elements in each chunk

⇒ array telling starting index of each chunk

⇒ sendcounts : { 1, 3, 2 } : for one master

⇒ displs : { 0, 2, 5 } and two slaves.

so array . . . A B C D E F G
rank 0 1 2

- Gather: All processes send chunks to root, root combines them into an array (rank order).

⇒ opposite of scatter.

MPI_Gather (...)

↳ // same as scatter.

- Like scatter root gathers from itself too, other things are similar to scatter as well e.g only root's recvbuff matter, others can be NULL, all chunks must be equal size, send-count is ~~count~~ no. of elements in a chunk, so is recvcount. (both are eq).

Gatherv is similar,

MPI_Gatherv (sendbuff, sendcount, sendtype, recvbuff, recvcounts, displs, recvtype, root, comm).

⇒ recvcount : how many each rank sends.

⇒ displs : where each chunk goes.

e.g recvcounts = { 1, 3, 2 }

displs = { 0, 2, 5 }

so arr . . . A Garbage C D E F G
rank 0 1 2

⑥ Synchronization.

• barrier.

a barrier is a synchronization points, all processes catch up here (all become as slow as the slowest process).

MPI-Barrier(comm)

↳ e.g. MPI_COMM_WORLD

• wait for all processes to reach here.

MPI-Alltoall(...)

↳ same as scatter

• every process does a scatter to everyone & gathers from everyone.

e.g. Rank 0 sends [A0 A1 A2]
" 1 " [B0 B1 B2] sending/
" 2 " [C0 C1 C2] scattering.
↓ ↓ ↓
Send to 0 1 2

Rank 0 got [A0 B0 C0] Gathering/
" 1 " [A1 B1 C1] receiving.
" 2 " [A2 B2 C2]

• essentially a matrix transpose.

MPI-Allgather(...)

↳ same as scatter.

• All send their data, everyone gets all of it.

e.g. rank 0 sends {0, 1}, all get
" 1 " {2, 3} {0, 1, 2, 3, 4, 5}
" 2 " {4, 5} ∴ personal copy.

- even though rank 0 and 1 are both in the master node, 0 is the actual master, 1 still acts like a slave like the rest.

(Node = machine)

- MPI_Send (data-buffer, size, type, dest, tag, comm)
 - ⇒ type MPI_INT, MPI_CHAR, MPI_DOUBLE
 - ⇒ dest rank of receiver
 - ⇒ tag - discussed before (MPI_COMM)
 - ⇒ Comm The communicator e.g MPI_COMM_WORLD.

- MPI_Recv (data-buffer, size, type, source, tag, comm, status)
 - source replaces dest here.
 - status is the metadata (MPI_Status)

MPI_Init (&arg, &argv);

// code

MPI_Finalize(); Isend and Irecv

- Send & Recv are blocking, so deadlock issue, we can fix it by:

```

MPI_Request req;
MPI_Isend (buffer, count, MPI_INT, dest, tag, MPI_COMM_WORLD,
// stuff                                     &request)
// more stuff.

MPI_Wait (&request, &status) // waits for send to complete.
                             same for recv.
  
```

- MPI_Probe can be used to get size of msg (peek msg w/o receiving it) (later).